

Paradigmas de Programación (M)

Introducción a la materia

1er cuatrimestre de 2025

Departamento de Computación

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Presentación de la materia

Repaso

Introducción a la programación funcional

Docentes (Turno Mañana)

Profesor

- ▶ Christian Cossio-Mercado

Jefes de trabajos prácticos

- ▶ Malena Ivnisky
- ▶ Brian Cardiff
- ▶ Daniel Grimaldi (sólo los viernes)

Ayudantes de segunda

- ▶ Jonathan Bekenstein
- ▶ Facundo Chenlo
- ▶ Magali Kruel
- ▶ Rafael Romani
- ▶ Ramiro Sánchez Posadas

Materiales

Todo el material de la materia va a estar disponible en el campus:
<https://campus.exactas.uba.ar/course/view.php?id=159>

- ▶ Diapositivas de las clases
- ▶ Guías de ejercicios
- ▶ Apuntes
- ▶ Enunciados de los trabajos prácticos
- ▶ Calendario
- ▶ Normativa
- ▶ ...

Revisen la sección “**útil**”.

Vías de comunicación

Docentes → alumnxs

Avisos a través del campus.

Alumnxs → docentes

Lista de correo: `plp-docentes@dc.uba.ar`

(para consultas administrativas)

Discusión entre estudiantes fuera del horario de la materia

Servidor de Discord: `https://tinyurl.com/plpdiscord`

(con eventual participación de docentes)

Algunos contenidos EXTRAS que daremos en las teóricas

- ▶ Además de los contenidos teórico-conceptuales de la materia veremos algunas cosas más en las teóricas.
- ▶ En cada clase les daremos algunas pistas y ayudas para poder sacar el máximo provecho de la materia. Por ejemplo:
 - ▶ ¿Cómo trabajo en grupo para los TPs?
 - ▶ ¿Cómo me conviene trabajar con las guías de ejercicios?
 - ▶ ¿Cómo me preparo para los parciales?
 - ▶ ¿Cómo hago si tengo que reentregar un TP?
 - ▶ ¿Cómo hago si tengo que recuperar un parcial?
 - ▶ ¿Qué hago con herramientas del estilo ChatGPT?
- ▶ Serán contenidos breves, pero esperamos que les sirvan para que su experiencia con la materia sea la mejor.

Discusión

- ▶ Nombre de la materia: “Paradigmas de programación”.
- ▶ ¿Y qué tienen que ver las preguntas que les pedimos completar?
 - ▶ ¿Qué es *programar*?
 - ▶ ¿Qué es un *programa*?
 - ▶ ¿Qué es un *algoritmo*?
 - ▶ ¿Qué es un *lenguaje de programación*?
 - ▶ ¿Qué es un *paradigma de programación*?
- ▶ ¿Qué temas creen que vamos a ver en la materia?
- ▶ ¿Por qué vemos esos y no otros?
- ▶ ¿Tiene algo que ver con el *enfoque* de la carrera?

Objeto de estudio de esta materia

Programación

Proceso de escribir instrucciones que una computadora puede ejecutar para resolver algún problema.

Programa

Una serie de instrucciones/definiciones que una computadora sigue para realizar una tarea específica.

¿Cómo se escriben esos programas?

Lenguajes de Programación

Lenguaje de Programación

Un formalismo artificial en el que se pueden describir computaciones.

Luego

La materia **Paradigmas de Programación** tiene como objetivo el estudio de los **fundamentos de los lenguajes de programación**.

Algunos Lenguajes de Programación I

- ▶ **1940 Plankalkul**: considerado el primer lenguaje de programación (Konrad Zuse)
- ▶ **1949 Lenguaje de Ensamblador** y **Shortcode**
- ▶ **1957 FORTRAN** Primer lenguaje de programación **de alto nivel** (John Backus)
- ▶ **1958 LISP** (List processor) Computación simbólica para IA (John McCarthy)
- ▶ **1969 ALGOL 60**: primer uso de la estructura de bloques, procedimientos recursivos, notación BNF, paso de parámetros por valor y por nombre, y varias otras características muy comunes en los lenguajes actuales
- ▶ **1959 COBOL** (Common Business Oriented Language)) Aplicaciones bancarias (Grace Hopper)
- ▶ **1967 Simula 67** Simulación, corutinas y clases.

Algunos Lenguajes de Programación II

- ▶ **1968 ALGOL 68** Estructuras de datos definidas por el usuario.
- ▶ **1970 PL/1** Primer intento a gran escala de un lenguaje con un amplio espectro de áreas de aplicación (IBM)
- ▶ **Principios de los 70 BASIC** (Beginners All-Purpose Symbolic Instruction Code) Fácil de aprender.
- ▶ **Principios de los 70 PASCAL** Fácil de aprender (Niklaus Wirth)
- ▶ **1972 C** Originalmente para programación de sistemas, Unix (Dennis Ritchie)
- ▶ **1972 Smalltalk** Programación orientada a objetos (Alan Kay, Adele Goldberg y Dan Ingalls)
- ▶ **1972 SQL** (Structured Query Language) Para bases de datos (Raymond Boyce y Donald Chamberlain)

Algunos Lenguajes de Programación III

- ▶ **Principios-mediados de los 70 PROLOG** Programación lógica (Alain Colmerauer y Philippe Roussel)
- ▶ **Mediados de los 70 Scheme** Dialecto de LISP, funciones como ciudadanos de primera clase.
- ▶ **Finales de los 70 ML** Propósito general, (principalmente) funcional (Robin Milner et al.)
- ▶ **1980 ADA** Sistemas críticos, encapsulación, abstracción de datos, excepciones, genericidad.
- ▶ **1983 C++** Ampliación de C con clases, plantillas, funciones virtuales (Bjarne Stroustrup)
- ▶ **1985 Eiffel** Orientado a objetos, Diseño por contrato (Bertrand Meyer)
- ▶ **1986 Erlang** Sistemas escalables, tiempo real, telecomunicaciones, modelo de actores (Joe Armstrong, Robert Virding, Mike Williams)

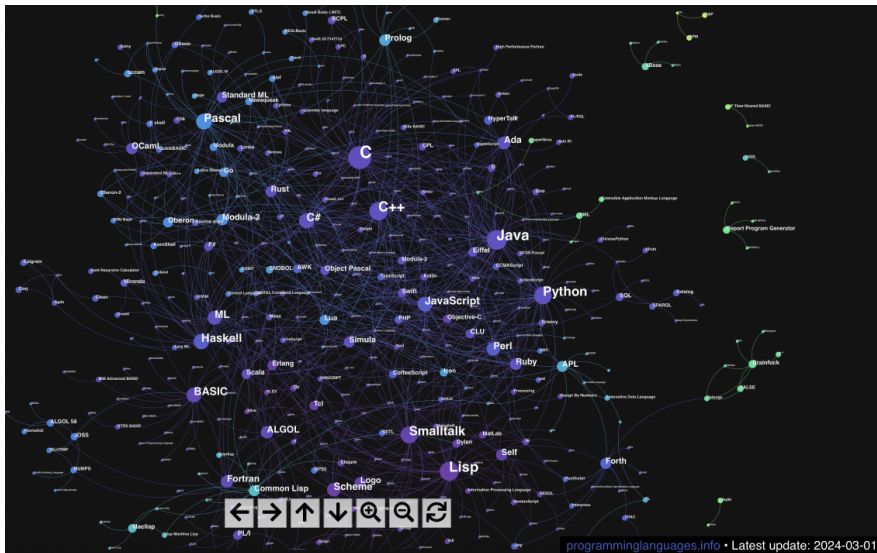
Algunos Lenguajes de Programación IV

- ▶ **1987 Perl** Procesamiento de texto y datos (Larry Wall)
- ▶ **1989 Coq** Programación funcional con tipos dependientes, asistencia para pruebas (Thierry Coquand et al.)
- ▶ **1990 Haskell** Programación funcional, inferencia de tipos.
- ▶ **1999 Agda** Programación funcional con tipos dependientes, asistencia para pruebas (Catarina Coquand)
- ▶ **1991 Python** Propósito general (Guido Van Rossum)
- ▶ **1993 Ruby** Aplicaciones web (Yukihiro Matsumoto)
- ▶ **1995 Java** Propósito general, (basado en clases) Orientado a objetos (James Gosling)
- ▶ **1995 JavaScript** Aplicaciones web, (basado en prototipos) Orientado a objetos (Brendan Eich)
- ▶ **1995 PHP** Aplicaciones web (Rasmus Lerdorf)
- ▶ **2003 Scala** Integra programación funcional y orientada a objetos (Martin Odersky)

Algunos Lenguajes de Programación V

- ▶ **2009 Go** Concurrency, comunicación por paso de mensajes (Robert Griesemer, Rob Pike y Ken Thompson)
- ▶ **2010 Rust** Programación de sistemas, multiparadigma, propiedad y préstamo (Graydon Hoare)
- ▶ **2014 Solidity** Smart contracts en Ethereum (Gavin Wood et al.)

Influencias



fuentes <https://programminglanguages.info/influence-network/>

¿Cómo vamos a tratar con la gran diversidad?

- ▶ Existen muchos lenguajes de programación:
 - ▶ La lista precedente contiene sólo a algunos de los lenguajes mas populares, o históricamente significativos.
- ▶ Sin embargo, hay características en ellos que son comunes.

Vamos a estudiar distintos enfoques o estilos para la construcción y estructuración de programas en un lenguaje de programación

Paradigma

Paradigma

Marco filosófico y teórico de una escuela científica o disciplina en la que se formulan teorías, leyes y generalizaciones y se llevan a cabo experimentos que les dan sustento

Fuente: Merriam-Webster¹

¹*A philosophical and theoretical framework of a scientific school or discipline within which theories, laws, and generalizations and the experiments performed in support of them are formulated*

Paradigma de Programación

Paradigma de programación

Marco filosófico y teórico en el que se formulan soluciones a problemas de naturaleza algorítmica

- ▶ Lo entendemos como
 - ▶ un estilo de programación
 - ▶ en el que se escriben soluciones a problemas en términos de algoritmos

Paradigmas de programación

- ▶ Paradigmas clásicos:
 - ▶ Imperativo
 - ▶ **Funcional**
 - ▶ **Lógico**
 - ▶ **Orientado a Objetos**
- ▶ Existen otros: concurrente, event-driven, cuántico, probabilístico, etc.

¿Qué estudiamos de un Lenguaje de Programación?

Diferentes Aspectos

- ▶ Gramática
- ▶ Semántica
- ▶ Pragmática
- ▶ Implementación

Gramática

El nivel que responde a la pregunta “¿Qué frases son correctas?”

- ▶ Establece
 - ▶ el **alfabeto**
 - ▶ Ejemplo: alfabeto latino de 22 o 26 letras, alfabeto cirílico, alfabeto chino, etc.
 - ▶ las **palabras** (o *tokens*): secuencia válida de símbolos
 - ▶ la **sintaxis**: las secuencias de palabras que son frases legales.

Semántica

El nivel que responde a la pregunta “¿**Qué significa una frase correcta?**”

- La semántica asigna un significado a cada frase correcta.

Pragmática

El nivel que responde a la pregunta “**¿Cómo usamos una frase significativa?**”

- ▶ Las frases con el mismo significado pueden usarse de diferentes maneras.
- ▶ Diferentes contextos pueden requerir diferentes frases:
 - ▶ algunas son más elegantes,
 - ▶ algunas son más eficientes,
 - ▶ algunas son más dialectales, etc.

Implementación

Implementación

El nivel que responde a la pregunta **“¿Cómo ejecutar una frase correcta, de manera que respetemos la semántica?”**

- ▶ Esto generalmente no es necesario para el usuario (programador)
- ▶ Es fundamental para los diseñadores e implementadores del lenguaje.

¿Qué aspectos estudiamos en Paradigmas de Programación?

- ▶ **Pragmática:** Vamos a usar tres lenguajes
 - ▶ Haskell, Prolog, Smalltalk
- ▶ **Semántica:** Estudiamos distintas alternativas para definir el significado de un programa de **manera formal**.
- ▶ **(Algunos aspectos de) Implementación:** Interpretación e inferencia de tipos.

Sobre sintaxis

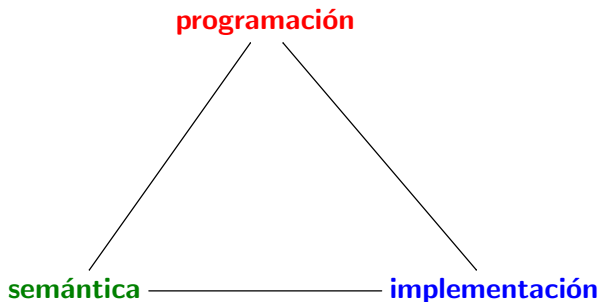
Vamos a hacer uso, pero este aspecto es objeto de estudio de **Lenguajes Formales, Autómatas y Computabilidad**

Cronograma

Programación funcional	2 semanas
Razonamiento ecuacional	1 semana
Lógica proposicional	1 semana
Cálculo- λ	2 semanas
(Repaso / consultas)	
Primer parcial	
Unificación e inferencia de tipos	1 semana
Lógica de primer orden	1 semana
Resolución	1 semana
Programación lógica	1,5 semanas
Programación orientada a objetos	1 semana
(Repaso / consultas)	
Segundo parcial	

Contenidos

Tres aspectos de los **lenguajes de programación**:



Motivación: programación

Los lenguajes de programación tienen distintas **características**.

- ▶ Etiquetado dinámico vs. tipado estático.
- ▶ Administración manual vs. automática de memoria.
- ▶ Funciones de primer orden vs. funciones de orden superior.
- ▶ Mutabilidad vs. inmutabilidad.
- ▶ Alcance dinámico vs. estático.
- ▶ Resolución de nombres temprana vs. tardía.
- ▶ Inferencia de tipos.
- ▶ Determinismo vs. no determinismo.
- ▶ Pasaje de parámetros por copia o por referencia.
- ▶ Evaluación estricta (por valor) o diferida (por necesidad).
- ▶ Tipos de datos inductivos, co-inductivos, GADTs, familias dependientes.
- ▶ *Pattern matching*, unificación.
- ▶ Polimorfismo paramétrico.
- ▶ Subclasificación, polimorfismo de subtipos, herencia simple vs. múltiple.
- ▶ Estructuras de control no local.
- ▶ ...

Motivación: programación

Distintas características permiten abordar un mismo problema de distintas maneras.

$$\begin{array}{r} \text{CMXXIV} \\ + \text{MCXLI} \\ \hline \end{array}$$

$$\begin{array}{r} 924 \\ + 1141 \\ \hline \end{array}$$

$$C = \{(x, y) \mid x^2 + y^2 = r^2\} \quad C = \{(r \sin \theta, r \cos \theta) \mid 0 \leq \theta < 2\pi\}$$

```
r := 1
while n > 0 {
  r := r * n
  n := n - 1
}
```

```
foldl (*) 1 [1..n]
```

Motivación: **semántica**

Dependemos del *software* en aplicaciones críticas.

- ▶ Telecomunicaciones.
- ▶ Procesos industriales.
- ▶ Reactores nucleares.
- ▶ Equipamiento médico.
- ▶ Previsión meteorológica.
- ▶ Aeronáutica.
- ▶ Vehículos autónomos.
- ▶ Transacciones monetarias.
- ▶ Análisis de datos en ciencia o toma de decisiones.
- ▶ ...

Las fallas cuestan recursos monetarios y vidas humanas.

Motivación: **semántica**

¿Podemos confiar en que un programa hace lo que queremos?

¿Y si el programa está escrito por el enemigo?

¿Y si el programa está escrito por una IA?

Objetivo

- ▶ Probar teoremas sobre el comportamiento de los programas.
- ▶ ¿Cómo darle significado matemático a los programas?
- ▶ En AED vimos una manera de hacerlo (triplas de Hoare).
- ▶ En PLP veremos otras maneras de dar semántica.

Motivación: **implementación**

Una computadora física ejecuta programas escritos en un lenguaje.
(El “código máquina”).

¿Cómo es capaz de ejecutar programas escritos en otros lenguajes?

- ▶ Interpretación (o evaluación).
- ▶ Chequeo e inferencia de tipos.
- ▶ Compilación (traducción de un lenguaje a otro).

Bibliografía (no exhaustiva)

Lógica proposicional y de primer orden

Logic and Structure

D. van Dalen.

Semántica y fundamentos de la implementación

Introduction to the Theory of Programming Languages

J.-J. Lévy, G. Dowek. Springer, 2010.

Types and Programming Languages

B. Pierce. The MIT Press, 2002.

Programación funcional

Thinking functionally with Haskell

R. Bird. Cambridge University Press, 2015.

Programación lógica

Logic Programming with Prolog

M. Bramer. Springer-Verlag, 2013.

Programación orientada a objetos

Smalltalk-80 the Language and its Implementation

A. Goldberg, D. Robson. Addison-Wesley, 1983.

Consideraciones

- ▶ Cronograma y actividades afectadas por paros: Los paros tendrán efectos sobre la cursada
- ▶ Esta edición está pensada para el plan 2023!

Presentación de la materia

Repaso

Introducción a la programación funcional

Repaso de Haskell

Definir las siguientes funciones:

- ▶ `factorial :: Int -> Int`
dado un entero $n \geq 0$, devuelve $n!$.
- ▶ `sumaN :: Int -> [Int] -> [Int]`
dado un entero k y una lista xs , devuelve la lista que resulta de sumarle k a cada elemento de xs .
- ▶ `aparece :: Int -> [Int] -> Bool`
dado un elemento x y una lista xs , devuelve un booleano que indica si x aparece en xs .

Más en general:

`aparece :: Eq a => a -> [a] -> Bool`

- ▶ `ordenar :: [Int] -> [Int]`
dada una lista, devuelve su permutación ordenada.

Más en general:

`ordenar :: Ord a => [a] -> [a]`

Repaso de Haskell

Definamos en Haskell las siguientes funciones:

- ▶ Dado el siguiente tipo de datos:

```
data Direccion = Norte | Este | Sur | Oeste
```

definir la función

```
opuesta :: Direccion -> Direccion
```

que dada una dirección d , devuelve la dirección opuesta a d .

- ▶ Dados los siguientes tipos de datos:

```
data Maybe a = Nothing | Just a
data AB a = Nil | Bin (AB a) a (AB a)
```

definir la función

```
buscar :: Eq a => a -> AB (a, b) -> Maybe b
```

que dada una clave k y un árbol binario de pares clave/valor, devuelve el valor asociado a la clave k en caso de que exista.

Se asume que no hay claves repetidas.

No se asume ningún otro invariante sobre el árbol.

Presentación de la materia

Repaso

Introducción a la programación funcional

Programación funcional

Un problema central en computación es el de procesar información:



La programación funcional consiste en definir funciones y aplicarlas para procesar información.

Las “funciones” son verdaderamente funciones (parciales):

- ▶ Aplicar una función no tiene efectos secundarios.
- ▶ A una misma entrada corresponde siempre la misma salida.
- ▶ Las estructuras de datos son inmutables.

Las funciones son datos como cualquier otro:

- ▶ Se pueden pasar como parámetros.
- ▶ Se pueden devolver como resultados.
- ▶ Pueden formar parte de estructuras de datos.
(Ej. árbol binario en cuyos nodos hay funciones).

Programación funcional

Un programa funcional está dado por un conjunto de ecuaciones:

Ejemplo

```
longitud [] = 0
```

```
longitud (x : xs) = 1 + longitud xs
```

```
longitud [10, 20, 30]
≡ longitud (10 : (20 : (30 : [])))
= 1 + longitud (20 : (30 : []))
= 1 + (1 + (longitud (30 : [])))
= 1 + (1 + (1 + longitud []))
= 1 + (1 + (1 + 0))
= 1 + (1 + 1)
= 1 + 2
= 3
```

Expresiones

Las **expresiones** son secuencias de símbolos que sirven para representar datos, funciones y funciones aplicadas a los datos. (Recordemos: las funciones también son datos).

Una expresión puede ser:

1. Un **constructor**:

True False [] (:) 0 1 2 ...

2. Una **variable**:

longitud ordenar x xs (+) (*) ...

3. La **aplicación** de una expresión a otra:

ordenar lista
not True
not (not True)
(+) 1
((+) 1) (alCuadrado 5)

4. También hay expresiones de otras formas, como veremos. Las tres de arriba son las fundamentales.

Expresiones

Convenimos en que la aplicación es **asociativa a izquierda**:

$$\begin{aligned} f \ x \ y &\equiv (f \ x) \ y && \not\equiv f \ (x \ y) \\ f \ a \ b \ c \ d &\equiv (((f \ a) \ b) \ c) \ d \end{aligned}$$

Ejemplo

$$\begin{aligned} &[1, 2] \\ \equiv &1 : [2] \\ \equiv &(:) 1 [2] \\ \equiv &((:)) 1 [2] \\ \equiv &((:)) 1 (2 : []) \\ \equiv &((:)) 1 ((:)) 2 [] \\ \equiv &((:)) 1 (((:)) 2 []) \end{aligned}$$

Expresiones

Ejemplo

`sumarUno = (+) 1`

```
sumarUno (sumarUno 5)
= ((+) 1) (sumarUno 5)
≡ 1 + sumarUno 5
= 1 + ((+) 1) 5
≡ 1 + (1 + 5)
= 1 + 6
= 7
```

Tipos

Hay secuencias de símbolos que no son expresiones bien formadas.

Ejemplo

1,,2)f x(

Hay expresiones que están bien formadas pero no tienen sentido.

Ejemplo

True + 1

0 1

[[] , (+)]

Tipos

Un **tipo** es una especificación del invariante de un dato o de una función.

Ejemplo

```
99          :: Int
not         :: Bool -> Bool
not True    :: Bool
(+)         :: Int -> (Int -> Int)
(+) 1       :: Int -> Int
((+) 1) 2   :: Int
```

El tipo de una función expresa un **contrato**.

Condiciones de tipado

Para que un programa esté **bien tipado**:

1. Todas las expresiones deben tener tipo.
2. Cada variable se debe usar siempre con un mismo tipo.
3. Los dos lados de una ecuación deben tener el mismo tipo.
4. El argumento de una función debe tener el tipo del dominio.
5. El resultado de una función debe tener el tipo del codominio.

$$\frac{f :: a \rightarrow b \quad x :: a}{f \ x :: b}$$

Sólo tienen sentido los programas bien tipados.

No es necesario escribir explícitamente los tipos. (Inferencia).

Tipos

Convenimos en que “ $\rightarrow$ ” es **asociativo a derecha**:

$$\begin{aligned} a \rightarrow b \rightarrow c &\equiv a \rightarrow (b \rightarrow c) \quad \text{✗} \quad (a \rightarrow b) \rightarrow c \\ a \rightarrow b \rightarrow c \rightarrow d &\equiv a \rightarrow (b \rightarrow (c \rightarrow d)) \end{aligned}$$

Ejemplo

```
suma4 :: Int -> Int -> Int -> Int -> Int  
suma4 a b c d = a + b + c + d
```

Se puede pensar así:

```
suma4 :: Int -> (Int -> (Int -> (Int -> Int)))  
(((suma4 a) b) c) d = a + b + c + d
```

Polimorfismo

Hay expresiones que tienen más de un tipo.

Usamos *variables de tipo* a , b , c para denotar tipos desconocidos:

```
id    :: a -> a
[]    :: [a]
(:)   :: a -> [a] -> [a]
fst   :: (a, b) -> a
snd   :: (a, b) -> b
```

Ejemplo

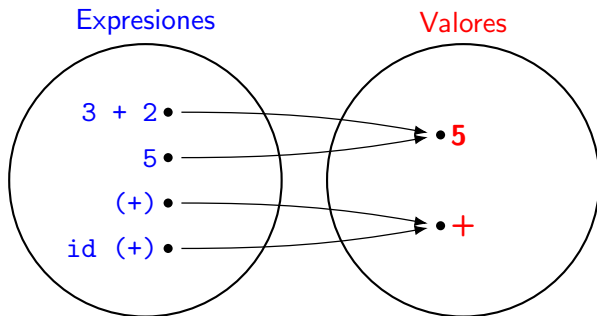
```
flip f x y = f y x
```

¿Qué tipo tiene flip?

```
flip (:) [2, 3] 1
= (:) 1 [2, 3]
≡ 1 : [2, 3]
= [1, 2, 3]
```

Modelo de cómputo

Dada una expresión, se computa su *valor* usando las ecuaciones:



Hay expresiones bien tipadas que no tienen valor. Ej.: $1 / 0$.

Decimos que dichas expresiones se indefinen o que tienen valor $\perp$.

Modelo de cómputo

Un programa funcional está dado por un conjunto de ecuaciones.
Más precisamente, por un conjunto de **ecuaciones orientadas**.

Una ecuación $e1 = e2$ se interpreta desde dos puntos de vista:

1. **Punto de vista denotacional.**

Declara que $e1$ y $e2$ tienen el mismo significado.

2. **Punto de vista operacional.**

Computar el valor de $e1$ se reduce a computar el valor de $e2$.

Modelo de cómputo

El lado *izquierdo* de una ecuación no es una expresión arbitraria. Debe ser una función aplicada a **patrones**.

Un patrón puede ser:

1. Una variable.
2. Un comodín `_`.
3. Un constructor aplicado a patrones.

El lado izquierdo no debe contener variables repetidas.

Ejemplo

¿Cuáles ecuaciones están sintácticamente bien formadas?

```
sumaPrimeros (x : y : z : _) = x + y + z
```

```
predecesor (n + 1) = n
```

```
iguales x x = True
```

Modelo de cómputo

Evaluar una expresión consiste en:

1. Buscar la subexpresión más externa que coincida con el lado izquierdo de una ecuación.
2. Reemplazar la subexpresión que coincide con el lado izquierdo de la ecuación por la expresión correspondiente al lado derecho.
3. Continuar evaluando la expresión resultante.

La evaluación se detiene cuando se da uno de los siguientes casos:

1. La expresión es un constructor o un constructor aplicado.

`True` `(:)` 1 `[1, 2, 3]`
2. La expresión es una función *parcialmente* aplicada.

`(+)` `(+) 5`
3. Se alcanza un *estado de error*.

Un estado de error es una expresión que no coincide con las ecuaciones que definen a la función aplicada.

Modelo de cómputo

Ejemplo: resultado — constructor

```
tail :: [a] -> [a]
```

```
tail (_ : xs) = xs
```

```
tail (tail [1, 2, 3])  $\rightsquigarrow$  tail [2, 3]  $\rightsquigarrow$  [3]
```

Ejemplo: resultado — función parcialmente aplicada

```
const :: a -> b -> a
```

```
const x y = x
```

```
const (const 1) 2  $\rightsquigarrow$  const 1
```

Modelo de cómputo

Ejemplo: indefinición — error

`head :: [a] -> a`

`head (x : _) = x`

`head (head [], [1], [1, 1])` $\rightsquigarrow$ `head []`
 $\rightsquigarrow \perp$

Ejemplo: indefinición — no terminación

`loop :: Int -> a`

`loop n = loop (n + 1)`

`loop 0` $\rightsquigarrow$ `loop (1 + 0)`
 $\rightsquigarrow$ `loop (1 + (1 + 0))`
 $\rightsquigarrow$ `loop (1 + (1 + (1 + 0)))`
 $\dots$

Modelo de cómputo

Ejemplo: evaluación no estricta

```
indefinido :: Int
```

```
indefinido = indefinido
```

```
    head (tail [indefinido, 1, indefinido])
```

```
  ~> head [1, indefinido]
```

```
  ~> 1
```

Modelo de cómputo

Ejemplo: listas infinitas

```
desde :: Int -> [Int]
```

```
desde n = n : desde (n + 1)
```

```
desde 0
```

```
≈ 0 : desde 1
```

```
≈ 0 : (1 : desde 2)
```

```
≈ 0 : (1 : (2 : desde 3)) ≈ ...
```

```
head (tail (desde 0))
```

```
≈ head (tail (0 : desde 1))
```

```
≈ head (desde 1)
```

```
≈ head (1 : desde 2)
```

```
≈ 1
```

Modelo de cómputo

Nota. En Haskell, el orden de las ecuaciones es relevante.
Si hay varias ecuaciones que coinciden se usa siempre la primera.

Ejemplo

```
esCorta (_ : _ : _) = False
esCorta _             = True
```

<code>esCorta []</code>	$\rightsquigarrow$	<code>True</code>
<code>esCorta [1]</code>	$\rightsquigarrow$	<code>True</code>
<code>esCorta [1, 2]</code>	$\rightsquigarrow$	<code>False</code>

Funciones de orden superior

Definamos la composición de funciones (“g . f”).

$$(\cdot) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c$$
$$(g \cdot f) \ x = g \ (f \ x)$$

Otra forma de definirla (usando la notación “lambda”):

$$(\cdot) :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c$$
$$g \cdot f = \lambda \ x \rightarrow g \ (f \ x)$$

Funciones de orden superior

¿Qué tienen en común las siguientes funciones?

```
dobleL :: [Float] -> [Float]
```

```
dobleL [] = []
```

```
dobleL (x : xs) = x * 2 : dobleL xs
```

```
esParL :: [Int] -> [Bool]
```

```
esParL [] = []
```

```
esParL (x : xs) = x `mod` 2 == 0 : esParL xs
```

```
longitudL :: [[a]] -> [Int]
```

```
longitudL [] = []
```

```
longitudL (x : xs) = length x : longitudL xs
```

Todas ellas siguen el esquema:

```
g [] = []
```

```
g (x : xs) = f x : g xs
```

Funciones de orden superior

¿Cómo se puede abstraer el esquema?

```
map :: (a -> b) -> [a] -> [b]
```

```
map f [] = []
```

```
map f (x : xs) = f x : map f xs
```

```
dobleL    xs = map (\ x -> x * 2) xs
```

```
esParL    xs = map (\ x -> x 'mod' 2 == 0) xs
```

```
longitudL xs = map length xs
```

Otra manera:

```
dobleL    = map (* 2)
```

```
esParL    = map ((== 0) . ('mod' 2))
```

```
longitudL = map length
```

Funciones de orden superior

¿Qué relación hay entre las siguientes funciones?

```
negativos :: [Int] -> [Int]
negativos []      = []
negativos (x : xs) = if x < 0
                      then x : negativos xs
                      else negativos xs
```

```
noVacias :: [[a]] -> [[a]]
noVacias []      = []
noVacias (x : xs) = if not (null x)
                      then x : noVacias xs
                      else noVacias xs
```

Ambas siguen el esquema:

```
g []      = []
g (x : xs) = if p x
              then x : g xs
              else g xs
```

Funciones de orden superior

¿Cómo se puede abstraer el esquema?

```
filter :: (a -> Bool) -> [a] -> [a]
filter p []          = []
filter p (x : xs) = if p x
                    then x : filter p xs
                    else filter p xs
```

```
negativos = filter (< 0)
noVacias  = filter (not . null)
```


Ejercicio

```
merge      :: (a -> a -> Bool) -> [a] -> [a] -> [a]  
mergesort  :: (a -> a -> Bool) -> [a] -> [a]
```

El primer parámetro es una función que determina una relación de orden total entre los elementos de tipo `a`.

ι ι ι ι ι ι ι ι ι ι ? ? ? ? ? ? ? ?

Lectura recomendada

Capítulo 4 del libro de Bird.

Richard Bird. *Thinking functionally with Haskell*.

Cambridge University Press, 2015.

Comentarios: tipos

Ojo. Dijimos:

“Cada variable se debe usar siempre con un mismo tipo.”

¿Está bien tipado el siguiente programa?

```
sucesor :: Int -> Int  
sucesor x = x + 1
```

```
opuesto :: Bool -> Bool  
opuesto x = not x
```

Sí. Hay dos “x” con distinto tipo pero son variables distintas.
El programa se podría reescribir así:

```
sucesor x = x + 1  
opuesto y = not y
```